Middle East Technical University

Department of Computer Engineering

CENG 336

Introduction to Embedded Systems Development

THE1

Due: 22th of March 2026, 23:59

Submission: via **ODTUClass**

1 Introduction

The purpose of this individual assignment is to familiarize you with the basic I/O operations and round-robin approach on PIC18F8722 using MPLAB X IDE simulation environment. To this end, you will implement a device that controls two independent LED outputs and a blinking indicator LED.

The program will use two 8-bit output ports connected to LEDs: one displaying a progress bar pattern and the other displaying a binary counter. A separate LED will blink independently at a fixed rate. Using a round-robin approach, the program will maintain precise timing for all outputs while handling the user input. The user will be able to pause/resume the progress bar and change the counting direction of the binary counter.

Figure 1 provides a high-level overview of the system. The program consists of three independent state machines that run concurrently in a round-robin loop. We recommend thinking about your solution in terms of these state machines.

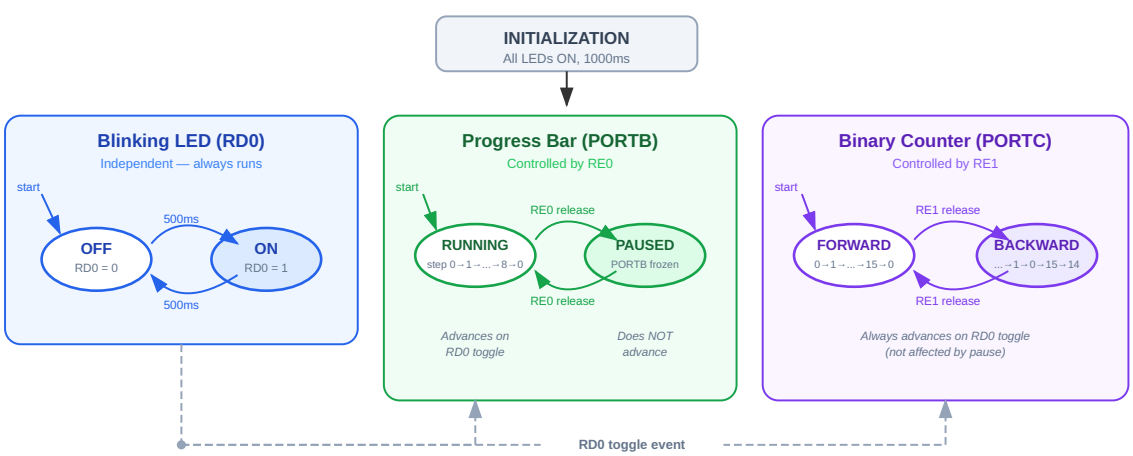

Figure 1: High-level state machine overview.

1.1 Notation

PORTX denotes an input/output port of PIC, where X is the letter of the port. LATX is the latch corresponding to PORTX. This assignment will use PORTB, PORTC, PORTD and PORTE. For a given PORTX, RXi for $0 \leq i \leq 7$ denotes the ith least significant bit in PORTX. For example, RB0 is the first (smallest) bit in PORTB. A single bit in a port is termed a "pin".

2 Specifications

2.1 Configuration

S-1 The program shall be written for **PIC18F8722** running at **1 MHz instruction frequency**.[1]

S-2 PORTB, PORTC and PORTD shall be used for output.

- PORTB shall display the progress bar pattern.
- PORTC shall display the binary counter value.
- RD0 shall be a blinking LED.

S-3 PORTE shall be used for input.

- RE0: Pause/resume the progress bar (but not the binary counter or the blinking LED).
- RE1: Change the counting direction of the binary counter.

2.2 Initialization

S-4 The program shall initialize all variables it uses, i.e., it shall not depend on variables being zero at the start.

S-5 The program shall light up all the LEDs in PORTB, PORTC and PORTD for 1000 ± 50 ms at the startup. This is called the **initialization period**.

S-5.1 The program does not need to respond to any inputs during this period, i.e., it can busy-wait. In this context, **busy-waiting** refers to a loop whose only purpose is to cause a time delay. During this loop, the program does not check for inputs and hence it's unresponsive.

S-6 After that, the program shall begin operation.

2.3 Blinking LED

S-7 Immediately after the initialization period, RD0 shall turn off for 500 ± 50 ms and then light up for 500 ± 50 ms, repeating this cycle indefinitely.

S-8 The blinking LED at RD0 shall continue regardless of the states of the progress bar and binary counter (i.e., it is not affected by pause/resume or direction changes).

[1] Note that the instruction frequency of the boards given to you is 10 MHz. Consequently, the program will run 10 times faster on that hardware. In this assignment, we decreased the instruction frequency to reduce the computational load of the simulation.

2.4 User Interaction

S-9 A "release" of a button is defined as a change from 1 to 0 in the corresponding pin.

S-9.1 Each pin is 0 by default, becomes 1 when pressed, and becomes 0 when released.

S-9.2 The program shall not busy-wait for the button release (1 to 0) after the button press (0 to 1). While the button is held (corresponding pin is 1), the program shall operate normally as if nothing happened. Only when a held button is released (1 to 0), a click is registered.

S-10 The program shall **NOT** implement button debouncing. Any change from 1 to 0 is a release, regardless of the duration or previous changes.

2.5 Progress Bar (PORTB)

S-11 The progress bar fills from the least significant bit to the most significant bit, one LED at a time. The pattern is:

$$\texttt{00000000} \to \texttt{00000001} \to \texttt{00000011} \to \texttt{00000111} \to \ldots \to \texttt{11111111} \to \texttt{00000000} \to \ldots$$

This is a 9-step cycle (including the empty state).

S-12 The progress bar shall advance one step each time RD0 changes state (every 500 ± 50 ms).

S-13 A button release in RE0 shall pause/resume the progress bar without affecting the binary counter or the blinking LED.

S-13.1 When paused, PORTB shall retain its current value and not advance.

S-14 After the initialization period, PORTB shall start from `00000000`.

2.6 Binary Counter (PORTC)

S-15 PORTC shall display a 4-bit binary counter (0 to 15) using the lower 4 bits (RC0–RC3). The upper 4 bits (RC4–RC7) shall always be 0.

S-16 The counter shall advance one step each time RD0 changes state (every 500 ± 50 ms).

S-17 The default counting direction is **forward** (incrementing).

S-17.1 In forward mode: $0 \to 1 \to 2 \to \ldots \to 15 \to 0 \to 1 \to \ldots$

S-17.2 In backward mode: $\ldots \to 2 \to 1 \to 0 \to 15 \to 14 \to \ldots$

S-18 A button release in RE1 shall toggle the counting direction (forward $\leftrightarrow$ backward).

S-19 The binary counter is **not affected** by the progress bar's pause/resume state. It always advances.

S-20 After the initialization period, PORTC shall start from `00000000` (displaying 0).

2.7 Implementation Details

S-21 The program shall use busy-waiting for the 1-second initialization period.

S-22 The program shall use a round-robin approach for the main operation and input handling.

S-23 Timers and interrupts are not allowed.

S-24 The program shall only write to the output ports when they need to change, i.e., once every 500 ± 50 ms. The program shall **NOT** perform unnecessary writes to the output ports in each iteration of the main loop. See Section 3.1 for details on why this is important.

3 Example Runs

In this section, example runs of the program are given in tables.

- ○ represents that the LED is off, i.e., the corresponding pin is 0.
- ● represents that the LED is lit, i.e., the corresponding pin is 1.
- 8 LEDs denoted by ○○○○○○○○ represent an 8-bit output port. The rightmost LED is the least significant bit. For example, ○○○○○○●● represents the decimal value of 3 on the corresponding output port.

The duration represents how long the given LED configuration is maintained before advancing to the next row. Comments and inputs are provided as special rows in each table.

Table 1: Example run with no input.

PORTB	PORTC	PORTD	Duration (ms)
●●●●●●●●	●●●●●●●●	●●●●●●●●	1000
Initialization period ends, main operation begins.			
○○○○○○○○	○○○○○○○○	○○○○○○○○	500
○○○○○○○●	○○○○○○○●	○○○○○○○●	500
○○○○○○●●	○○○○○○●○	○○○○○○○○	500
○○○○○●●●	○○○○○○●●	○○○○○○○●	500
○○○○●●●●	○○○○○●○○	○○○○○○○○	500
○○○●●●●●	○○○○○●○●	○○○○○○○●	500
○○●●●●●●	○○○○○●●○	○○○○○○○○	500
○●●●●●●●	○○○○○●●●	○○○○○○○●	500
●●●●●●●●	○○○○●○○○	○○○○○○○○	500
○○○○○○○○	○○○○●○○●	○○○○○○○●	500
○○○○○○○●	○○○○●○●○	○○○○○○○○	500
○○○○○○●●	○○○○●○●●	○○○○○○○●	500
○○○○○●●●	○○○○●●○○	○○○○○○○○	500
○○○○●●●●	○○○○●●○●	○○○○○○○●	500
○○○●●●●●	○○○○●●●○	○○○○○○○○	500
○○●●●●●●	○○○○●●●●	○○○○○○○●	500
○●●●●●●●	○○○○○○○○	○○○○○○○○	500
●●●●●●●●	○○○○○○○●	○○○○○○○●	500
The pattern continues indefinitely.			

Table 2: Pause/resume example using RE0.

PORTB	PORTC	PORTD	Duration (ms)
●●●●●●●●	●●●●●●●●	●●●●●●●●	1000
Initialization period ends, main operation begins.			
○○○○○○○○	○○○○○○○○	○○○○○○○○	500
○○○○○○○●	○○○○○○○●	○○○○○○○●	500
○○○○○○●●	○○○○○○●○	○○○○○○○○	200
A button release occurs at RE0. Progress bar is paused.			
○○○○○○●●	○○○○○○●○	○○○○○○○○	300
○○○○○○●●	○○○○○○●●	○○○○○○○●	500
○○○○○○●●	○○○○○●○○	○○○○○○○○	500
○○○○○○●●	○○○○○●○●	○○○○○○○●	300
A button release occurs at RE0. Progress bar is resumed.			
○○○○○○●●	○○○○○●○●	○○○○○○○●	200
○○○○○●●●	○○○○○●●○	○○○○○○○○	500
○○○○●●●●	○○○○○●●●	○○○○○○○●	500
The pattern continues indefinitely.			

Table 3: Direction change example using RE1.

PORTB	PORTC	PORTD	Duration (ms)
●●●●●●●●	●●●●●●●●	●●●●●●●●	1000
Initialization period ends, main operation begins.			
○○○○○○○○	○○○○○○○○	○○○○○○○○	500
○○○○○○○●	○○○○○○○●	○○○○○○○●	500
○○○○○○●●	○○○○○○●○	○○○○○○○○	200
A button release occurs at RE1. Counter direction changes to backward.			
○○○○○○●●	○○○○○○●○	○○○○○○○○	300
○○○○○●●●	○○○○○○○●	○○○○○○○●	500
○○○○●●●●	○○○○○○○○	○○○○○○○○	500
○○○●●●●●	○○○○●●●●	○○○○○○○●	500
○○●●●●●●	○○○○●●●○	○○○○○○○○	500
The pattern continues indefinitely.			

Table 4: Combined example: pause, direction change, and simultaneous button release.

PORTB	PORTC	PORTD	Duration (ms)
●●●●●●●●	●●●●●●●●	●●●●●●●●	1000
Initialization period ends, main operation begins.			
○○○○○○○○	○○○○○○○○	○○○○○○○○	500
○○○○○○○●	○○○○○○○●	○○○○○○○●	200
A button release occurs at RE0. Progress bar is paused.			
○○○○○○○●	○○○○○○○●	○○○○○○○●	300
○○○○○○○●	○○○○○○●○	○○○○○○○○	500
○○○○○○○●	○○○○○○●●	○○○○○○○●	300
A button release occurs at RE1. Counter direction changes to backward.			
○○○○○○○●	○○○○○○●●	○○○○○○○●	200
○○○○○○○●	○○○○○○●○	○○○○○○○○	500
○○○○○○○●	○○○○○○○●	○○○○○○○●	200
RE0 and RE1 are released simultaneously. Progress bar is resumed. Counter direction changes to forward.			
○○○○○○○●	○○○○○○○●	○○○○○○○●	300
○○○○○○●●	○○○○○○●○	○○○○○○○○	500
○○○○○●●●	○○○○○○●●	○○○○○○○●	500
The pattern continues indefinitely.			

3.1 Grading

Your submission will be tested using black-box tests. However, for technical reasons, your program needs to abide by the following rules for correct testing:

1. **DO NOT** perform unnecessary writes to the output ports in each iteration of the main loop. The tester uses the `watch` command to track changes in ports. If you keep rewriting data to the output ports in every iteration, the debugger will be overwhelmed and the test will fail.

2. When the simulator first runs, all variables will be set to 0. You program **MUST NOT** depend on this behavior. The tester works by performing a hardware reset after each test case. After a hardware reset, the variables will **NOT** be set to 0. If you assume that all variables are set to 0 at the start, it's likely that your program will exhibit peculiar behavior after the first test case.

3. Make sure that there are **no non-ASCII characters** (e.g. special Turkish characters) in your source code. Non-ASCII characters may cause issues during grading.

Please note that not abiding by these rules may result in **0 grade** due to complete testing failure, or a significant grade reduction.

The tentative grading rubric is given in Table 5. Additional test cases may be included during grading, so ensure that your implementation follows the specifications closely. Please keep in mind that this grading rubric does not include the late submission penalty (if applicable) and other external penalties that may incur due to cheating or using forbidden features (e.g. timers and interrupts).

Table 5: Tentative Grading Rubric.

Criterion	**Points**
Initialization period (2.2)	5.0
Baseline operation without input *Progress bar pattern, binary counter, blink timing, wrap-arounds*	25.0
Pause/Resume control – RE0 (2.5) *Pause, resume, pause again, resume again;* *counter & blink must continue while paused*	25.0
Direction change control – RE1 (2.6) *Forward→backward, backward→forward, backward wrap (0 → 15);* *progress bar & blink must continue*	25.0
Combined tests *Simultaneous button release, buttons at different times,* *button held behavior*	20.0
Total	**100.0**

In order to test your submission, please **go over the items in the grading rubric and confirm that each item works correctly in the simulator**. Inspect the PORT values in the debugger by putting breakpoints and use the stopwatch to verify the timing.

4 Regulations

1. This is not a group assignment, you must solve it **individually**.
2. Any clarifications and revisions to the assignment will be posted to ODTUClass. You are expected to follow the announcements so that you are up to date.
3. Ask your questions in the discussion forum dedicated for this assignment in ODTUClass. Unless you have *specific* question about *your code*, use the forum. If you think that your question is too specific to ask on the forum, you can ask your questions via email to Halil: `hcandan@ceng.metu.edu.tr`
4. Please consult your lecture notes and datasheets first before asking any questions.
5. **Submission:** Upload a single `*.s` file to the ODTUClass assignment. The file name does not matter as long as the extension is correct. Your source file must include your **name** and **student ID** as comments at the top.
6. **Late Policy:** You can extend the deadline by 3 days at maximum. If you extend the deadline by n days, you will receive a grade penalty of $5n^2$.
7. Please keep in mind that this is an individual assignment. You are not allowed to reveal your solution to other group members in any way, shape, or form, even partially.
8. **Using Large Language Models (LLMs) and other AI tools:** We note that AI can be a useful learning resource but if it is used for generating code that you are supposed to write, it can have a detrimental effect on your learning. Therefore, **using code from other resources, whether it is generated by humans or AI, is considered cheating**.
9. Do **not** enable the extended instruction set (`XINST`). It is beyond the scope of this assignment and will cause problems during testing.
10. Please **use the provided starting project**. Otherwise, you may encounter problems and lose points due to incorrect configuration.
11. Your submission will be tested on the **department lab computers (ineks)**. Make sure your program compiles and runs correctly on that environment before submitting.

5 Hints

1. There are plenty of resources available to you in ODTUClass. To refer to instructions and specifications of PIC18F8722, you should use the PIC18F8722 Datasheet. Since you will be using MPLAB X IDE simulation environment for this assignment, Recitation 1 documents can also be a great set up guide. If you set up a local environment, you are recommended to use MPLAB X IDE version 5.45 and XC8 Compiler version 2.30, both are available in the downloads archive. If you cannot set up a local environment, you can use the department labs.

2. MPLAB IDE X v5.45 direct download links:
   - Linux
   - Windows
   - macOS
3. XC8 Compiler v2.30 direct download links:
   - Linux
   - Windows
   - macOS
4. **Stopwatch** tool of the simulator can be used to measure time spent by a code segment, by adding **breakpoints** to the start and end of that segment. You can find this window from **Window** → **Debugging** menu. It is important to configure your instruction frequency to 1 MHz from **Project properties** → **Simulator** → **Instruction Frequency** before starting the simulator for the time values to be correct. (1 MHz is the default but double-check to make sure.) **Project Properties** window can be reached by right clicking the project name in **Projects** panel. You can refer to Recitation 1 documents for detailed explanations and screenshots.
5. You can also see the states of your ports, variables and pins by using the Logical Analyzer, SFR and Variables windows. They can be found under the **Window** menu in **Simulator, Debugging, Target Memory Views** sections respectively.
6. Your main loop should be short enough to detect button releases reliably. If your loop takes too long, the program may miss a button release entirely.
7. Do **not** remove the `resetVec` section from the starting project template. It is required for the program to handle hardware resets properly, which is essential during testing.

5.1 Troubleshooting

1. **pic-as is not installed:** pic-as should be installed automatically alongside XC8 Compiler v2.30. If it's not recognized, please check if the `pic-as` folder is present under the XC8 installation directory. If this folder is present, you can add pic-as manually using the instructions in Figure 2. The default XC8 installation directories are as follows:
   - Linux: `/opt/microchip/xc8/v2.30/`
   - Windows: `C:\Program Files\Microchip\xc8\v2.30\`
2. **Cannot fire stimulus:** When you fire an event in the stimulus window, you should see a notification like "Asynchronous stimulus RB0 fired" at the bottom of the window. If it doesn't appear and the stimulus doesn't happen (note that the stimulus becomes effective only after you step/continue the debugger), this issue might be caused by **non-ASCII characters in your source code** or the encoding. Your code must NOT contain any non-ASCII characters. If the issue persists after removing all non-ASCII characters, try replacing your `.s` file with either `main.s` provided in the starting project to ensure that the file has the right encoding.

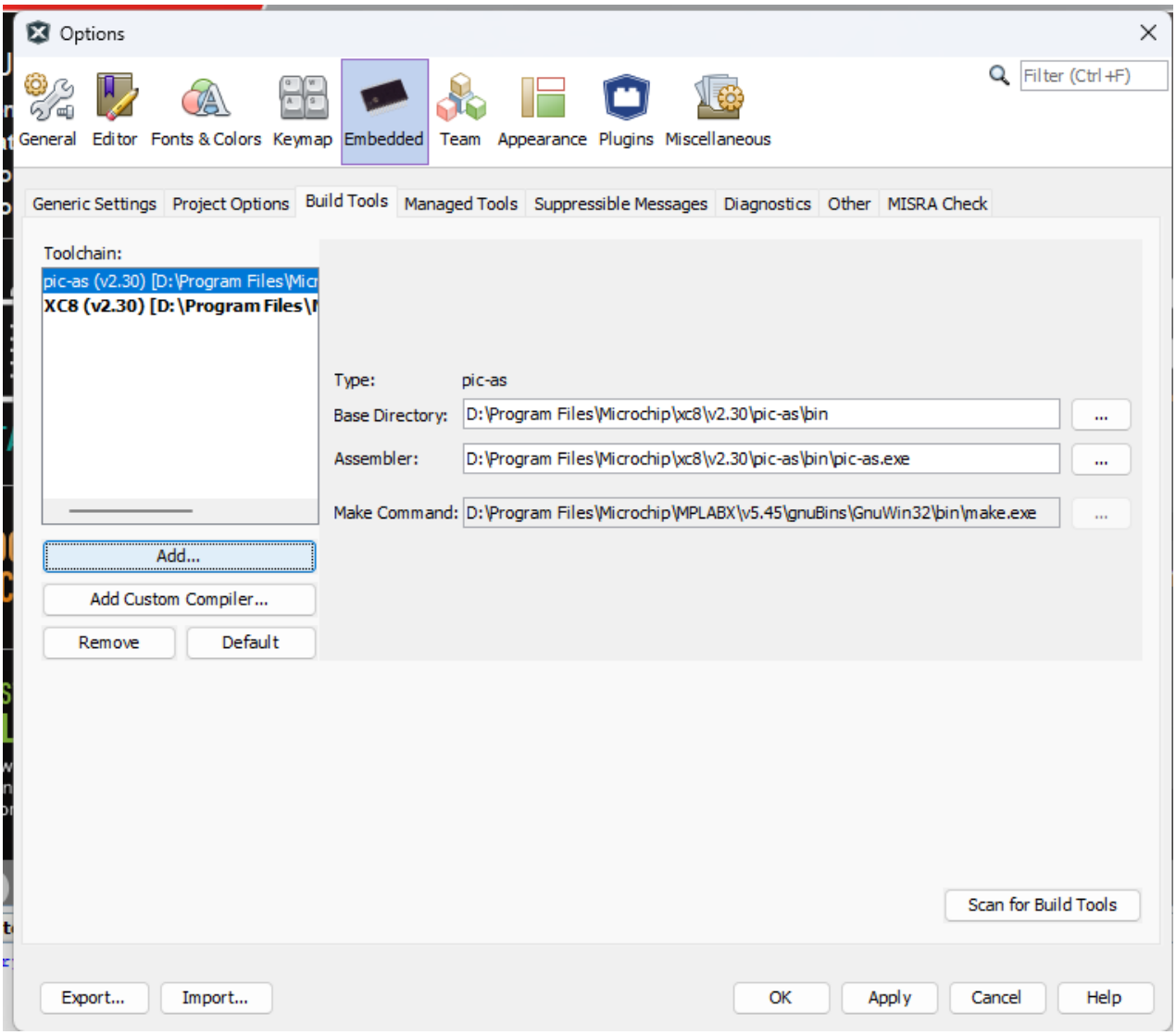

Figure 2: Adding pic-as manually.